

¿Qué es el *deep learning*?

1) IA, *machine learning* y *deep learning*

- **IA (inteligencia artificial)**: paraguas amplio de técnicas para automatizar tareas "intelectuales".
- **Machine learning (ML)**: subcampo de la IA donde el sistema **aprende a partir de ejemplos** en vez de ser programado con reglas explícitas.

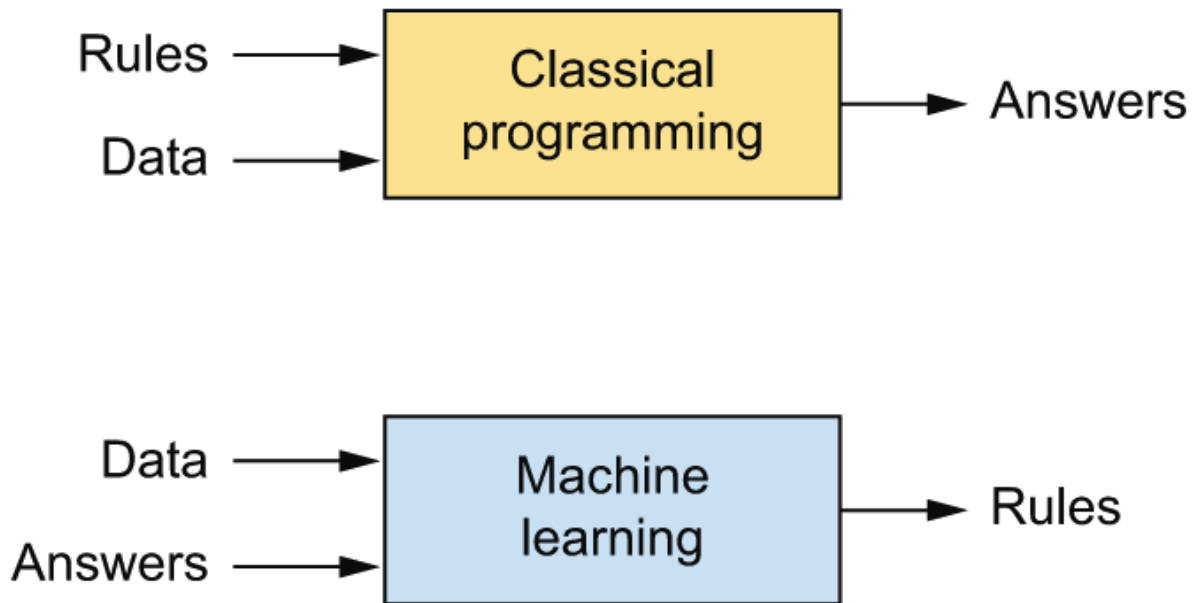


Figure 1.2: Machine learning: a new programming paradigm

- **Deep learning (DL)**: subcampo del ML que aprende representaciones **en muchas capas** (profundidad), típicamente con **redes neuronales**.

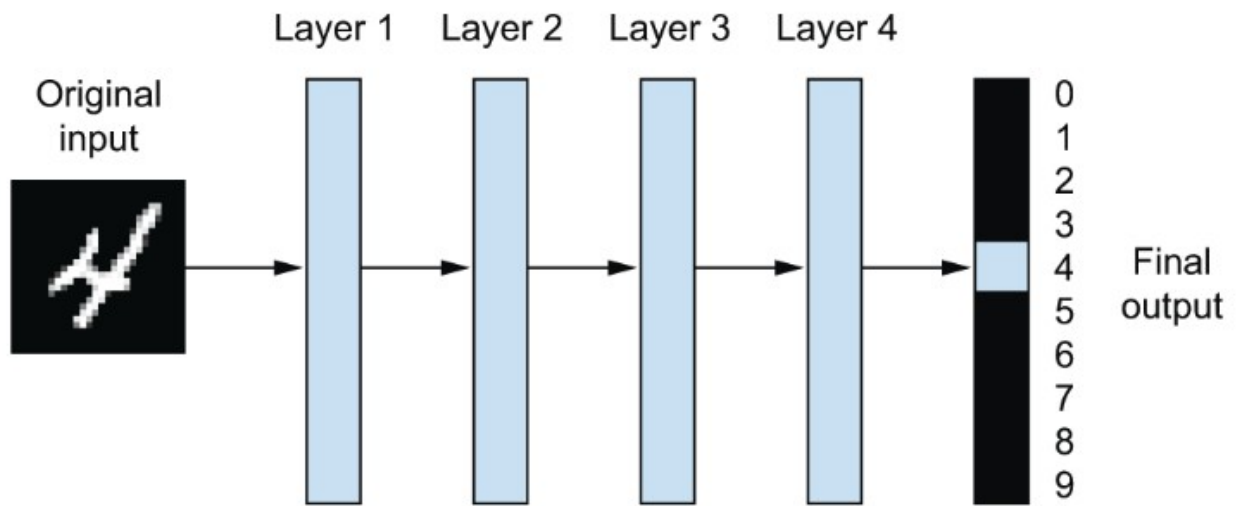
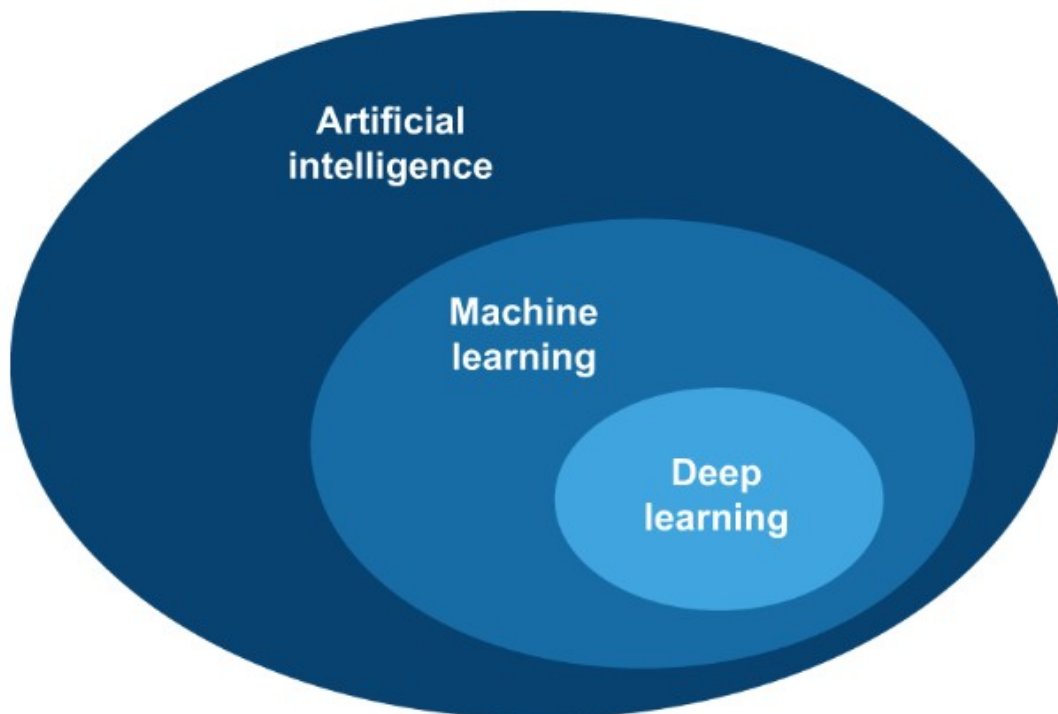


Figure 1.5: A deep neural network for digit classification

Idea clave: ML se apoya en datos + una señal de evaluación (métrica o función de pérdida) para ir ajustando el comportamiento del modelo.



2) Aprender reglas y representaciones a partir de datos

Un sistema de ML suele necesitar:

1. **Datos de entrada** (por ejemplo, imágenes o audio).

2. **Objetivos/etiquetas** (por ejemplo, "gato"/"perro", o transcripciones).
3. **Una medida de error** para saber si lo está haciendo bien (p. ej., % de acierto o una *loss*).

En vez de diseñar manualmente reglas complejas, el modelo busca transformaciones de los datos que hagan el problema más sencillo. Esas transformaciones son **representaciones** (formas de codificar/transformar la información).

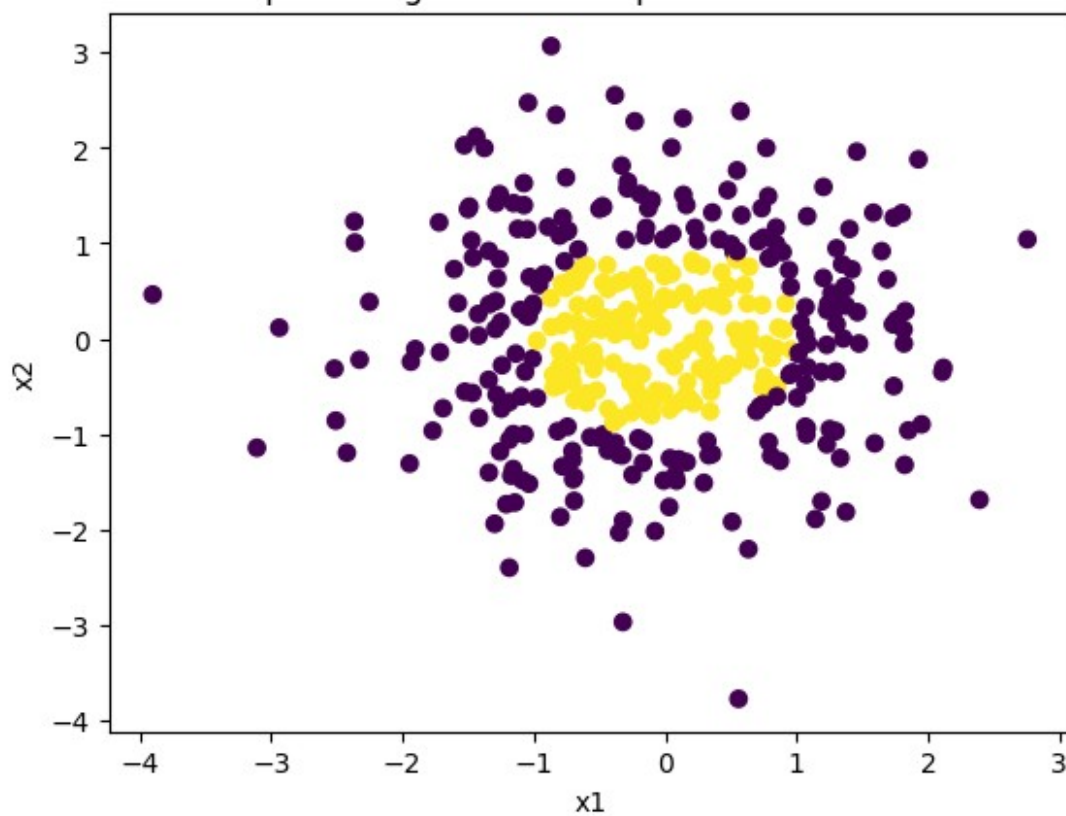
```
# Ejemplo didáctico: cambiar de representación en un problema 2D
# (ilustra la idea de que una transformación puede volver separable un
problema)
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(0)
n = 400
X = rng.normal(size=(n, 2))
# Etiqueta: puntos dentro de un círculo -> 1, fuera -> 0 (no lineal en
coordenadas originales)
y = (np.sum(X**2, axis=1) < 1.0).astype(int)

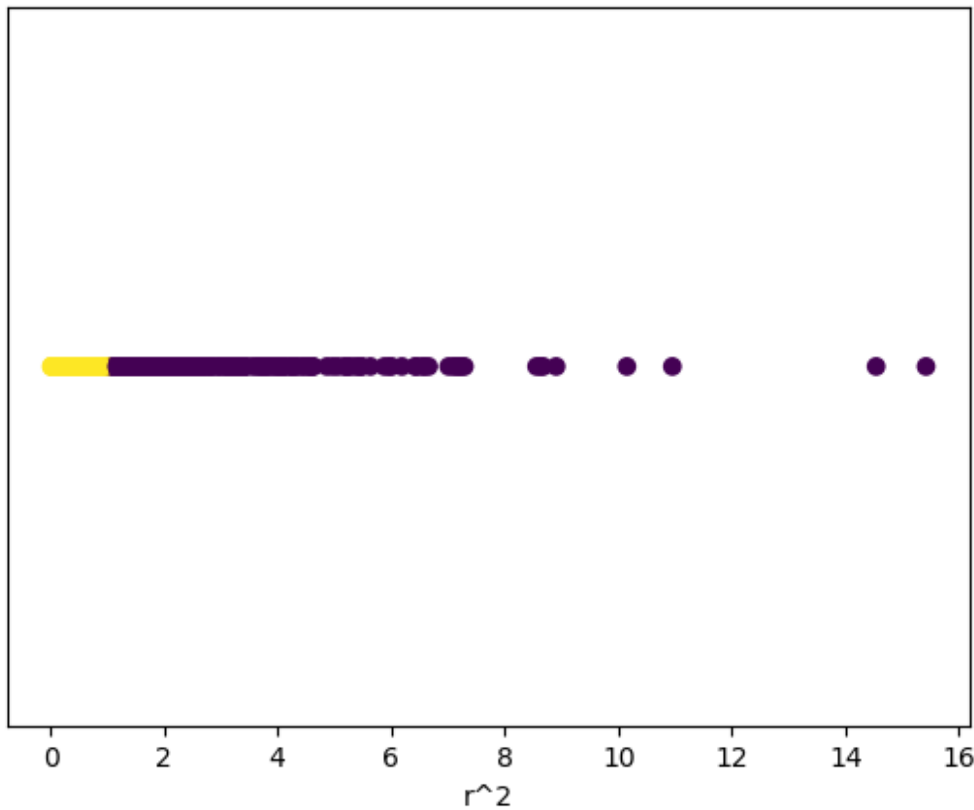
plt.figure()
plt.scatter(X[:,0], X[:,1], c=y)
plt.title('Espacio original: difícil separar con una recta')
plt.xlabel('x1'); plt.ylabel('x2')
plt.show()

# Nueva representación:  $r^2 = x_1^2 + x_2^2$  (una sola característica)
r2 = np.sum(X**2, axis=1)
plt.figure()
plt.scatter(r2, np.zeros_like(r2), c=y)
plt.title('Nueva representación:  $r^2$  separa casi con un umbral')
plt.xlabel('r^2'); plt.yticks([])
plt.show()
```

Espacio original: difícil separar con una recta



Nueva representación: r^2 separa casi con un umbral



3) Qué significa lo "profundo" en *deep learning*

"Profundo" no significa que el modelo "entienda" de forma filosófica. Significa que aprende una **secuencia de capas de representación**: cada capa transforma la salida de la anterior y, en conjunto, el sistema aprende representaciones cada vez más útiles para la tarea.

Comparación típica:

- En enfoques "poco profundos" (*shallow*), suele haber 1–2 transformaciones y mucha **ingeniería de características** manual.
- En *deep learning*, muchas de esas características se **aprenden automáticamente** dentro de la red.

Aunque el término "red neuronal" viene por analogías históricas, en la práctica se trata de un marco matemático/ingenieril de optimización.

4) Cómo aprende una red neuronal (intuición)

En términos prácticos, entrenar una red suele tener estos ingredientes:

- **Capas con parámetros (pesos)**: números que determinan la transformación.
- **Función de pérdida (loss)**: cuantifica el error entre la predicción y el objetivo.

- **Optimizador:** regla que ajusta los pesos para reducir la pérdida. La idea central es calcular cómo cambia la pérdida respecto a cada peso (gradientes) y actualizar en la dirección que la disminuye.

Ese ajuste repetido sobre muchos ejemplos (a menudo en mini-lotes) es el **bucle de entrenamiento**.

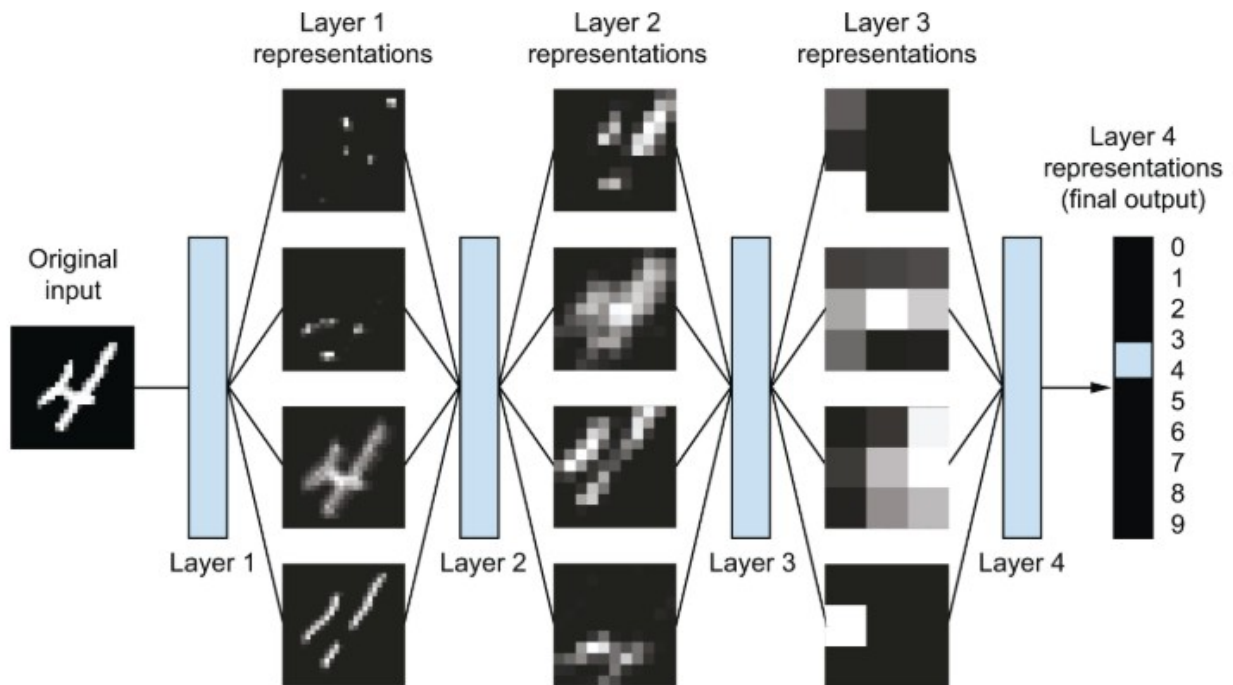


Figure 1.6: Deep representations learned by a digit-classification model

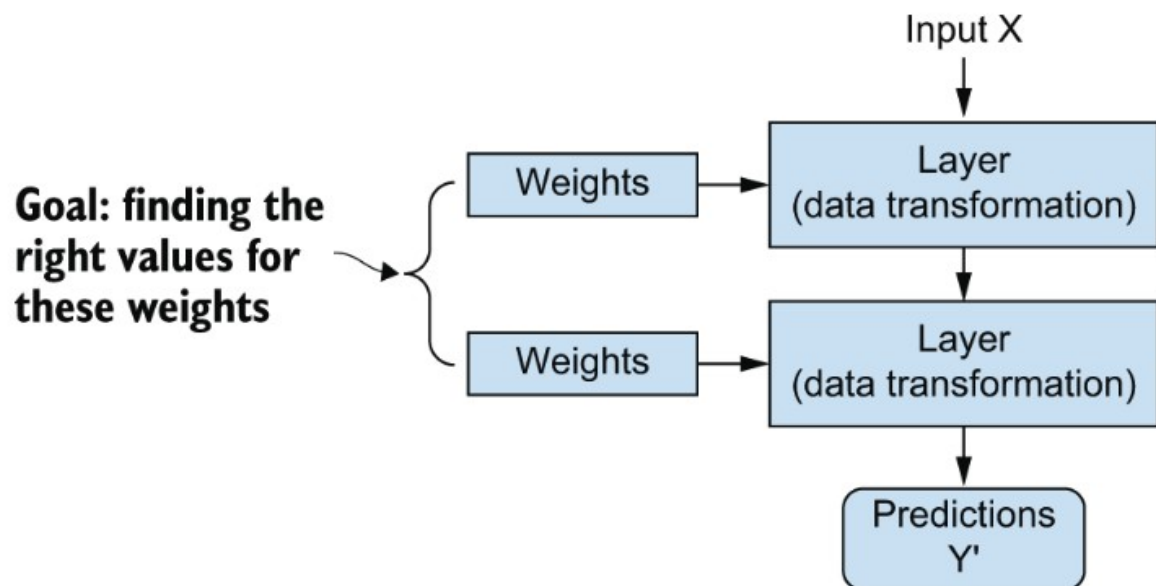


Figure 1.7: A neural network is parameterized by its weights.

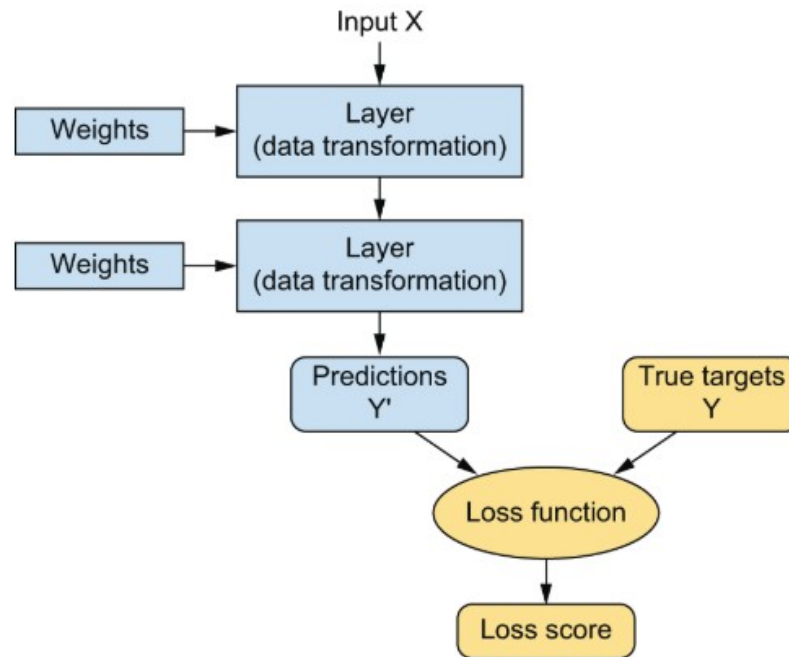


Figure 1.8: A loss function measures the quality of the network's output.

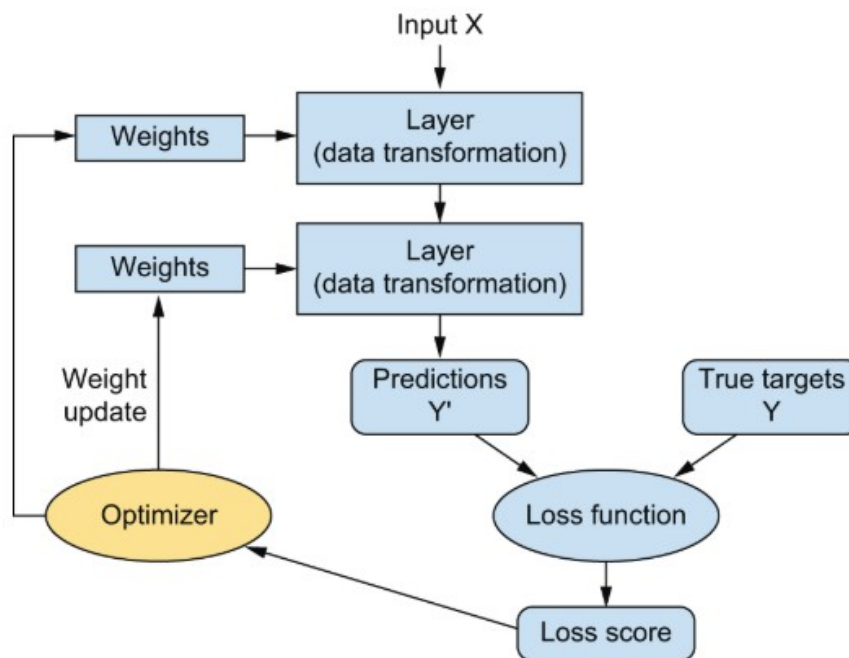


Figure 1.9: The loss score is used as a feedback signal to adjust the weights.

5) ¿Qué hace diferente al *deep learning*?

En el capítulo se resaltan (al menos) estas ideas:

1. **Simplicidad del pipeline:** al aprender representaciones automáticamente, reduce la dependencia de ingeniería manual de características y puede reemplazar tuberías complejas por modelos end-to-end.
2. **Escalabilidad:** se beneficia mucho de hardware paralelo (GPU/TPU) y del entrenamiento por mini-lotes.
3. **Reutilización:** modelos entrenados pueden adaptarse a nuevas tareas (transfer learning), y la noción moderna de **modelos fundacionales** explota esta idea a gran escala.

6) La era de la IA generativa (contexto)

El capítulo menciona la ola reciente de sistemas generativos (chatbots y generación de imágenes) como ejemplo visible del impacto del deep learning.

Puntos de contexto (en términos generales):

- Mucho del progreso reciente se apoya en **escala** (datos, cómputo, parámetros) y en técnicas que reducen cuellos de botella de etiquetado manual.
- A la vez, conviene mantener una postura crítica: distinguir avances reales de *hype* a corto plazo.

7) Ideas para llevarte

- Deep learning = **aprendizaje de representaciones por capas**.
- Entrenar = **minimizar una pérdida** ajustando parámetros con un optimizador.
- La potencia práctica viene de: automatizar *features*, escalar con hardware y reutilizar modelos.

Referencia

- Capítulo original (inglés): https://deeplearningwithpython.io/chapters/chapter01_what-is-deep-learning/